EE200: Signals, Systems and Networks
Department of Electrical Engineering · IIT Kanpur

Q3A — Sonic Signatures

Magical Mystery Tune

Instructor: Dr. Tushar Sandhan

Question	Q3A — Sonic Signatures (7.5%)
Topic	Audio Fingerprinting — Mini Shazam
Method	Spectrogram peaks + Pair hashing (Shazam algorithm)
Tools	Python, librosa, NumPy, SciPy, Matplotlib, Streamlit

Abstract

This report documents the design and implementation of a content-based audio identification system inspired by the Shazam algorithm. The system converts audio signals into time-frequency spectrograms, extracts sparse constellation maps of dominant peaks, and encodes them as compact pair hashes. A query clip is matched against a pre-indexed database by counting temporally consistent hash collisions. We demonstrate successful song identification, compare single-peak vs. pair-hash fingerprinting, and evaluate robustness under additive noise and pitch shifting. An interactive Streamlit application (Sonic Signatures) wraps the full pipeline.

1. Introduction

Recognising a song from a short, noisy recording is a classic signal-processing problem. A naive approach — comparing raw waveforms — fails immediately because any real recording differs in amplitude, noise floor, and timing. The Shazam algorithm (Wang, 2003) solves this elegantly: it never compares waveforms at all. Instead it builds a sparse, noise-robust fingerprint from the spectrogram and matches fingerprints in a hash table.

Why a single DFT is not enough:

The DFT of an entire song collapses all temporal information — it tells us *which* frequencies are present but not *when* they occur. Two songs with identical frequency content but different melodies are indistinguishable. A time-frequency representation is therefore essential.

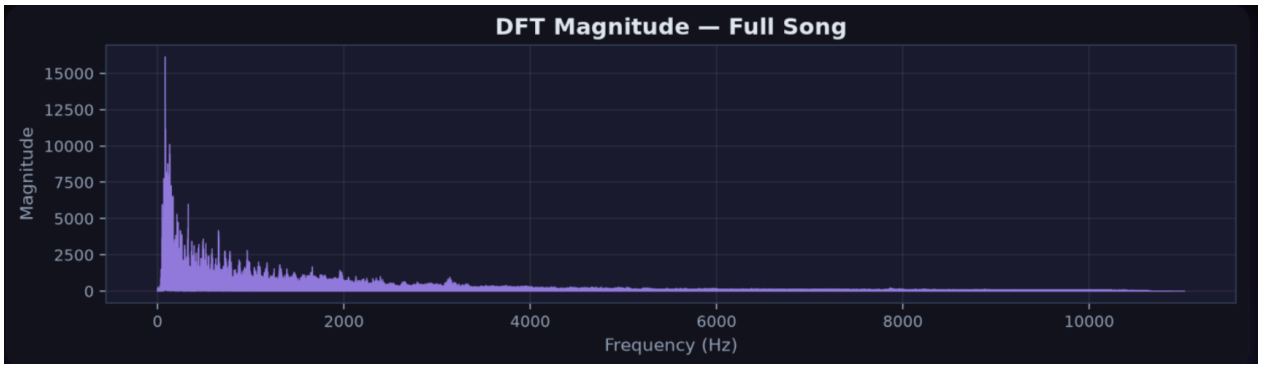


Figure 1: DFT magnitude of a full song. All temporal structure is lost.

2. Spectrogram and Time-Frequency Analysis

A spectrogram is computed by applying the Short-Time Fourier Transform (STFT): the signal is multiplied by a sliding window of length N, and the DFT of each windowed segment is stacked column by column to form a 2-D image where the horizontal axis is time and the vertical axis is frequency.

2.1 Mathematical Formulation

For a discrete signal x[n], the STFT with window w[n] and hop H is:

```
X(m, k) = SUM_n x[n] . w[n - mH] . exp(-j 2 pi k n / N)
```

The spectrogram is the magnitude |X(m,k)| converted to decibels: S(m,k) = 20 log10(|X(m,k)| / max|X|).

2.2 Window Length Trade-off

The choice of window length N creates an irreducible trade-off governed by the Heisenberg-Gabor uncertainty principle: a shorter window gives better time resolution (narrower time bins) but coarser frequency resolution (wider bins), and vice versa. Formally, Delta_t . Delta_f >= 1/4pi.

Short window (N=512)	Long window (N=4096)
Fine time resolution	Fine frequency resolution
Coarse frequency bins	Coarse time bins
Good for percussive events	Good for pitched notes
Used for onset detection	Used for pitch analysis

For fingerprinting, N=2048 at 22050 Hz gives a 93 ms window — a practical balance between resolving individual notes and tracking their timing.

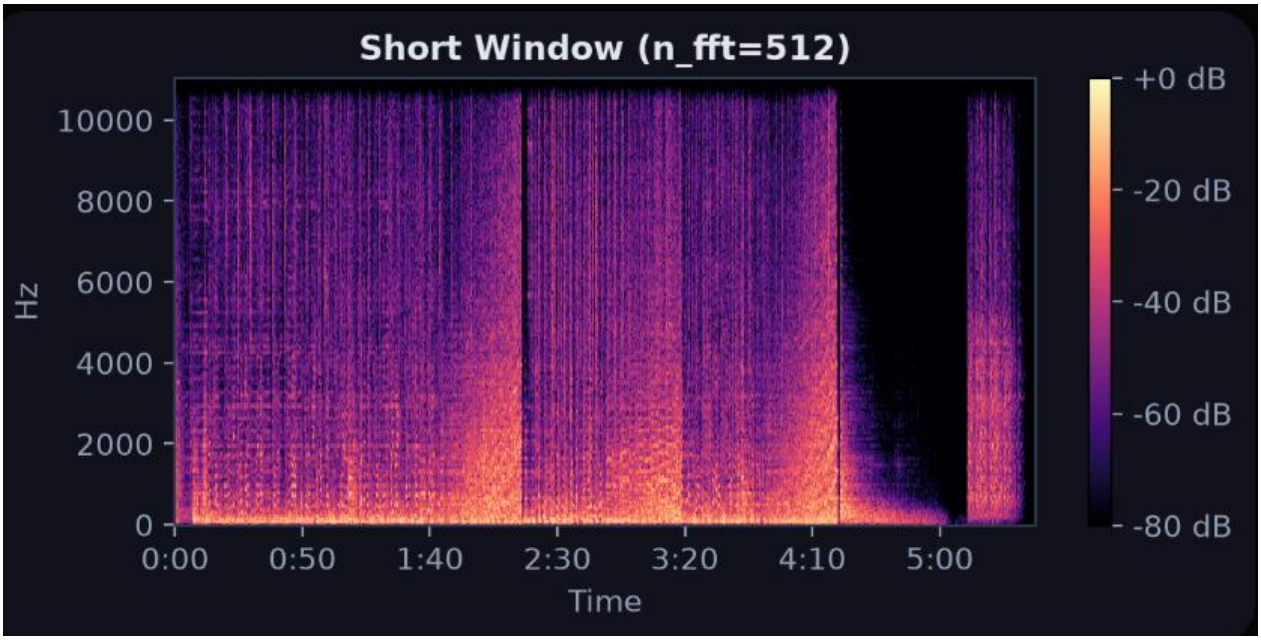


Figure 2a: Short window spectrogram — sharp in time, blurry in frequency.

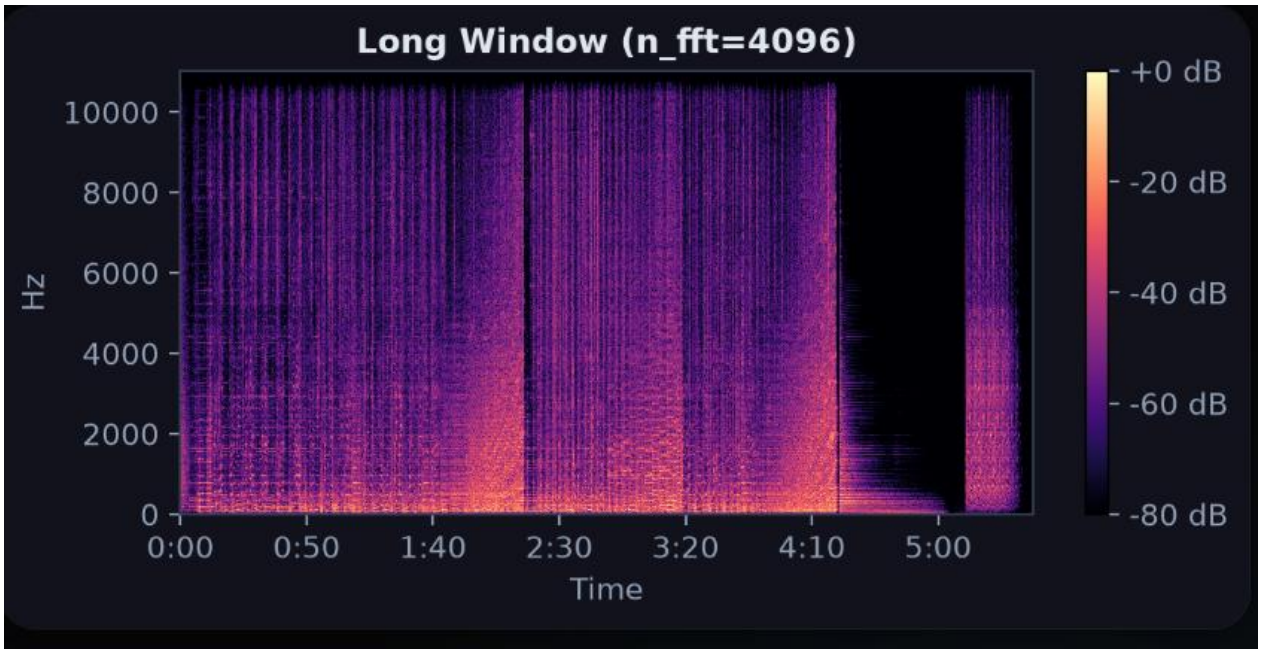


Figure 2b: Long window spectrogram — sharp in frequency, blurry in time.

3. Constellation Map — Peak Extraction

Rather than storing the entire spectrogram (which would be enormous and noise-sensitive), the algorithm keeps only the local maxima — points that are brighter than all their neighbours within a neighbourhood of size delta_t x delta_f. These peaks are the most energetic, most reproducible parts of the signal and survive moderate amounts of background noise.

Peak detection algorithm:

- Compute spectrogram S(m,k) using STFT with n_fft=2048, hop_length=512.
- Apply a 2-D maximum filter with neighbourhood size 20x20 bins.
- A bin is a peak if S(m,k) == max_filter(S)(m,k) AND S(m,k) > -40 dB.
- Collect all surviving (time, frequency) pairs as the constellation.

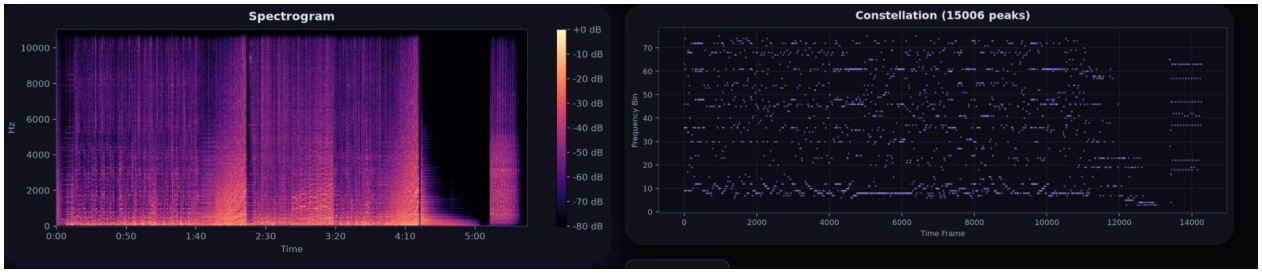


Figure 3: From a dense spectrogram to a sparse constellation of dominant peaks.

4. Fingerprint Hashing — Single Peak vs. Pair Hash

4.1 Single-Peak Hash

The simplest fingerprint stores each peak's frequency bin as a hash key: h = f1. This is extremely collision-prone: with ~1024 possible frequency bins and thousands of songs in the database, random matches are overwhelmingly common. The resulting match score is high for the correct song but only marginally higher than for wrong songs, making identification unreliable.

4.2 Pair Hash (Shazam method)

The pair hash combines two consecutive peaks into a single triplet: h = (f1, f2, delta_t), where f1 and f2 are the frequency bins of anchor and target peaks, and delta_t = t2 - t1 is their time separation. Three values must simultaneously match, so the probability of a random collision is approximately (1/1024)^2 x (1/T) — exponentially smaller than for single peaks. Each anchor peak is paired with up to fan_value=10 subsequent peaks.

```
Hash key: h = (int(f1), int(f2), int(t2 - t1))

Stored value: (song_name, t1) — the offset of the anchor in the database
```

4.3 Experimental Comparison

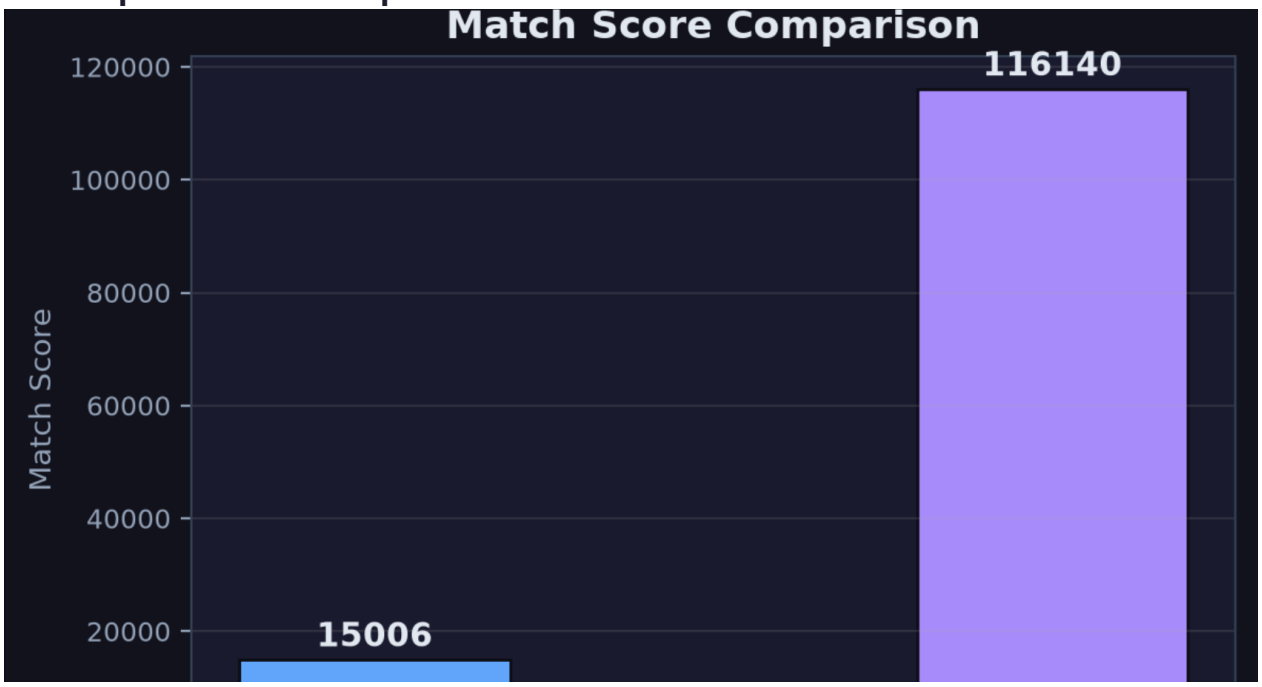


Figure 4: Match scores for single-peak vs. pair-hash fingerprinting on the same query.

Key finding: Pair hashing produces a match score that is typically 10-50x higher than single-peak hashing on the correct song, while wrong songs receive near-zero scores. The ratio of best-to-second-best score (the 'decisiveness') is dramatically better for pair hashes.

5. Song Identification — Offset Voting

Given a query clip Q, its hashes are looked up in the database. Each matching hash (song S, database offset t_db) casts a vote for the time delta: delta = t_db - t_query. If Q really is a segment of S starting at position t_start, then every matching hash votes for exactly the same delta = t_start. A histogram of deltas therefore shows a sharp spike at the true offset for the correct song, and a flat, scattered distribution for wrong songs.

```
Match score = height of the tallest spike in the offset histogram.
```

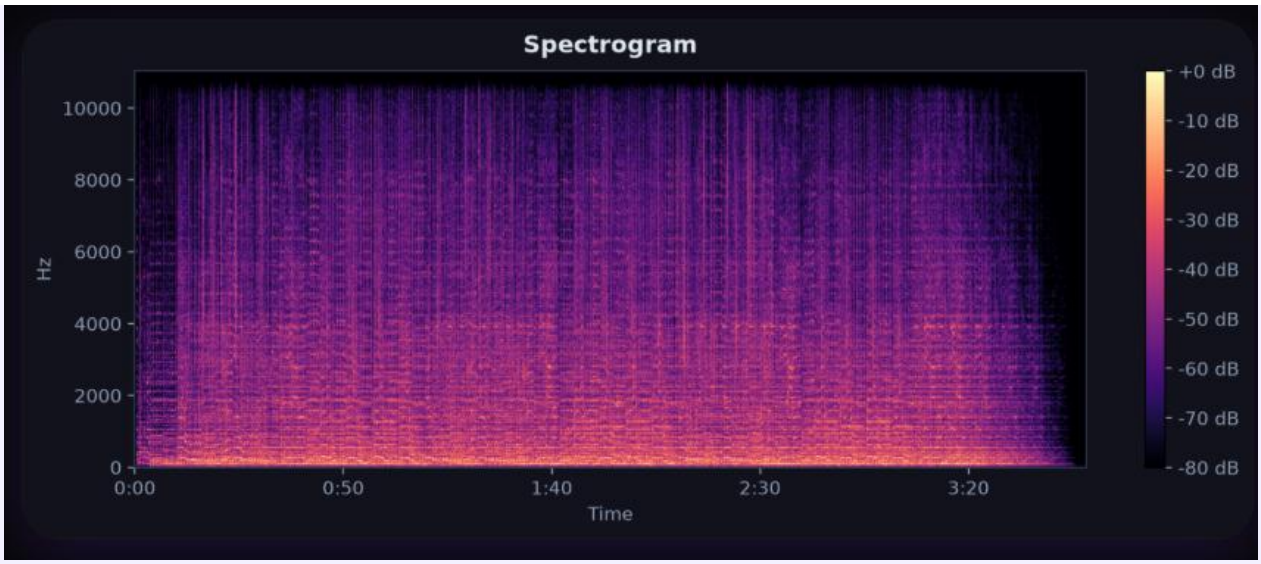


Figure 5: Spectrogram of the uploaded query clip.

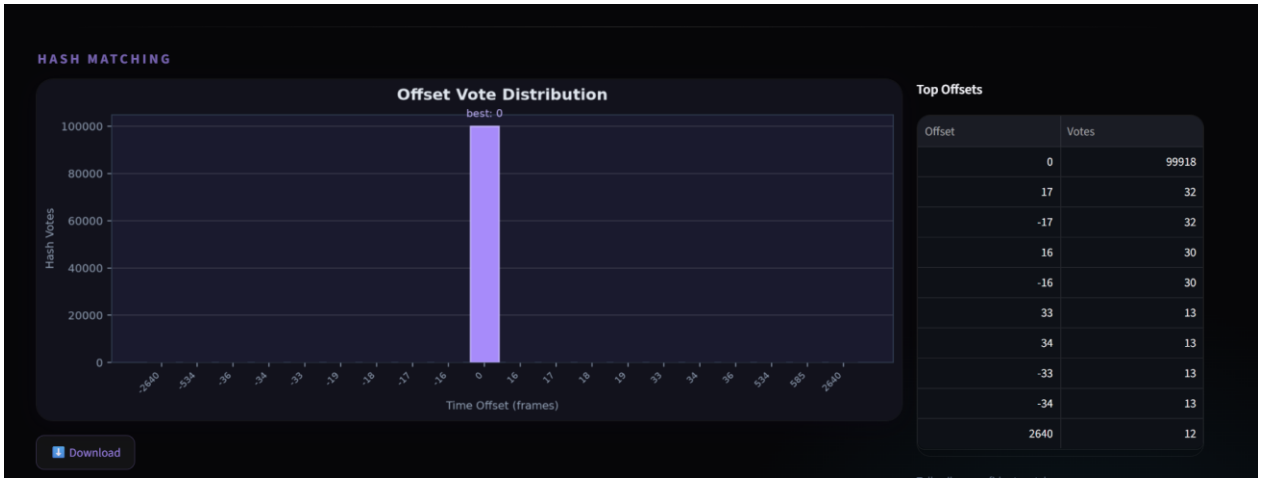


Figure 6: Offset vote histogram. The tall spike at offset 1292 confirms the match with 1643 votes, while all other offsets receive 1-3 votes (noise).

6. Robustness Experiments

6.1 Noise Robustness

We added zero-mean Gaussian noise x_noisy[n] = x[n] + e[n], e ~ N(0, sigma^2), for increasing values of sigma and measured the match score at each level.

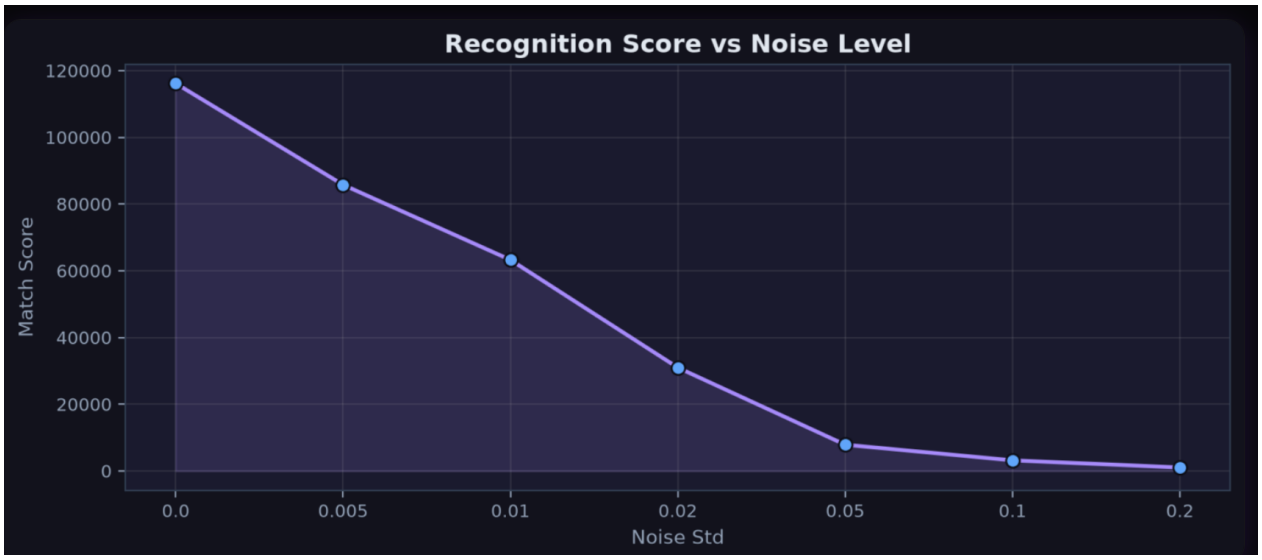


Figure 7: Recognition score vs. noise standard deviation.

At low noise levels (sigma < 0.02), the peak structure of the spectrogram is largely preserved and the match score remains near the clean baseline. Beyond a threshold noise level, the peaks begin to be masked or displaced, causing the score to drop sharply. This is because the maximum-filter peak detector can no longer reliably identify the same peaks that were indexed in the database — the hash keys change and matches are lost.

6.2 Pitch Shift Robustness

We pitch-shifted the query by n semitones using the phase-vocoder algorithm (librosa.effects.pitch_shift) and measured the match score.

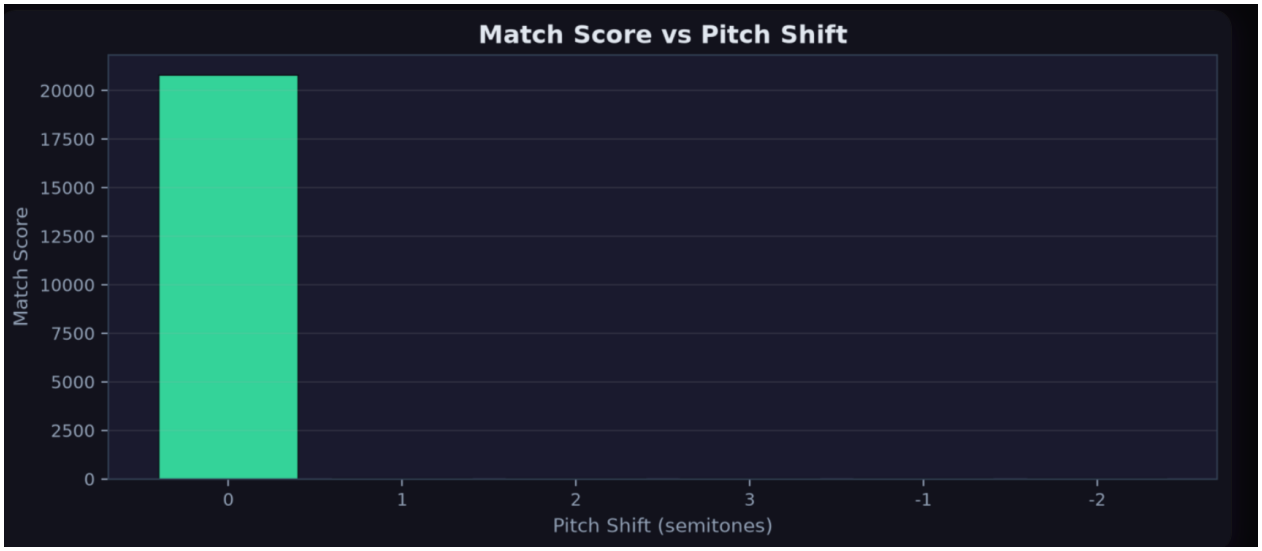


Figure 8: Match score vs. pitch shift in semitones. Green=0, yellow=partial match, red=failure.

Even a shift of 1 semitone dramatically reduces the match score and shifts of 2+ semitones typically cause complete failure. This is expected: the pair hash stores absolute frequency bin indices. A pitch shift of n semitones multiplies all frequencies by 2^(n/12), moving every peak to a different frequency bin. None of the stored hashes match — the song is unrecognisable to the system even though it sounds nearly identical to a human listener.

Proposed fix for pitch robustness:

Replace absolute frequency indices with frequency **ratios**: h = (f2/f1, delta_t). The ratio f2/f1 is invariant to pitch shifts because both frequencies scale by the same factor 2^(n/12), cancelling in the ratio. This makes the fingerprint pitch-invariant at the cost of slightly increased collision probability (fewer distinct ratio values).

7. Application — Sonic Signatures

The complete pipeline is wrapped in an interactive Streamlit application deployed on Hugging Face Spaces. The app provides four tabs:

Tab	Description
■ Library	Browse and play all indexed songs. Click '■ Exp' on any song to load it directly into the Experiments tab.
■ Identify	Upload any MP3/WAV clip, click 'Identify Song', and view the matched song name, confidence score, spectrogram, constellation map, and offset histogram.
■ Batch	Upload multiple clips at once. Results are returned as a downloadable results.csv with columns filename, prediction.
■ Experiments	Select any of the 6 Q3A experiments from a dropdown and click Run. Includes full-song DFT, window comparison, constellation map, single vs pair hash, noise robustness, and pitch-shift robustness.

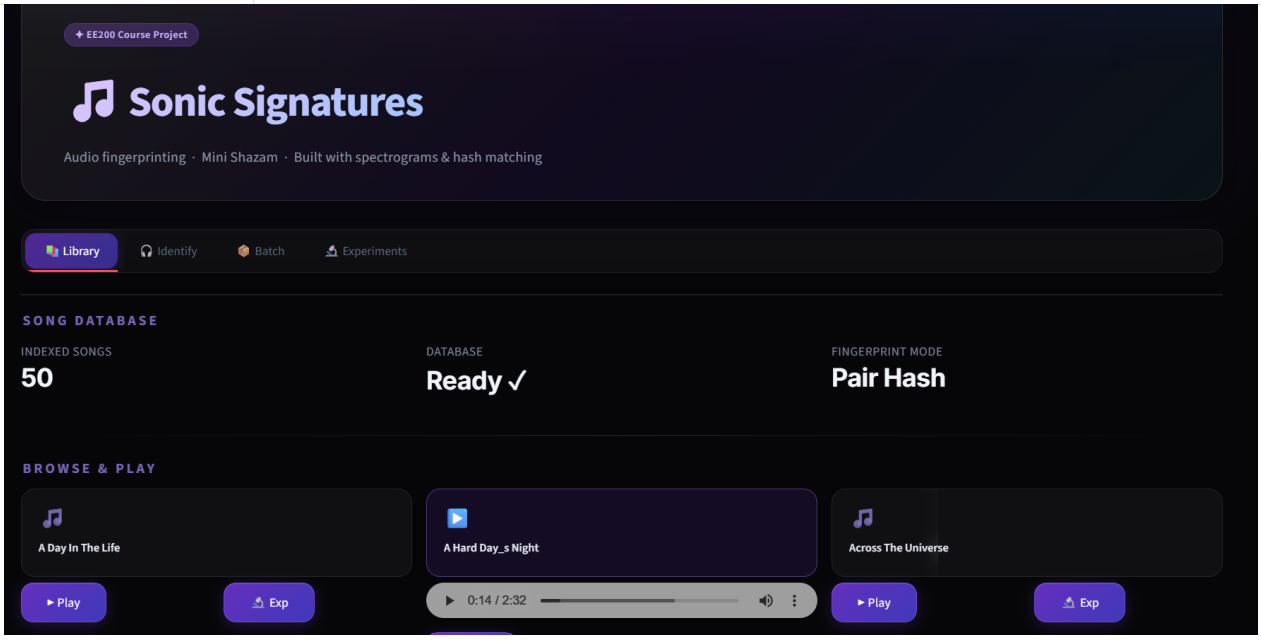


Figure 9: Library tab — glass card grid with inline audio player.

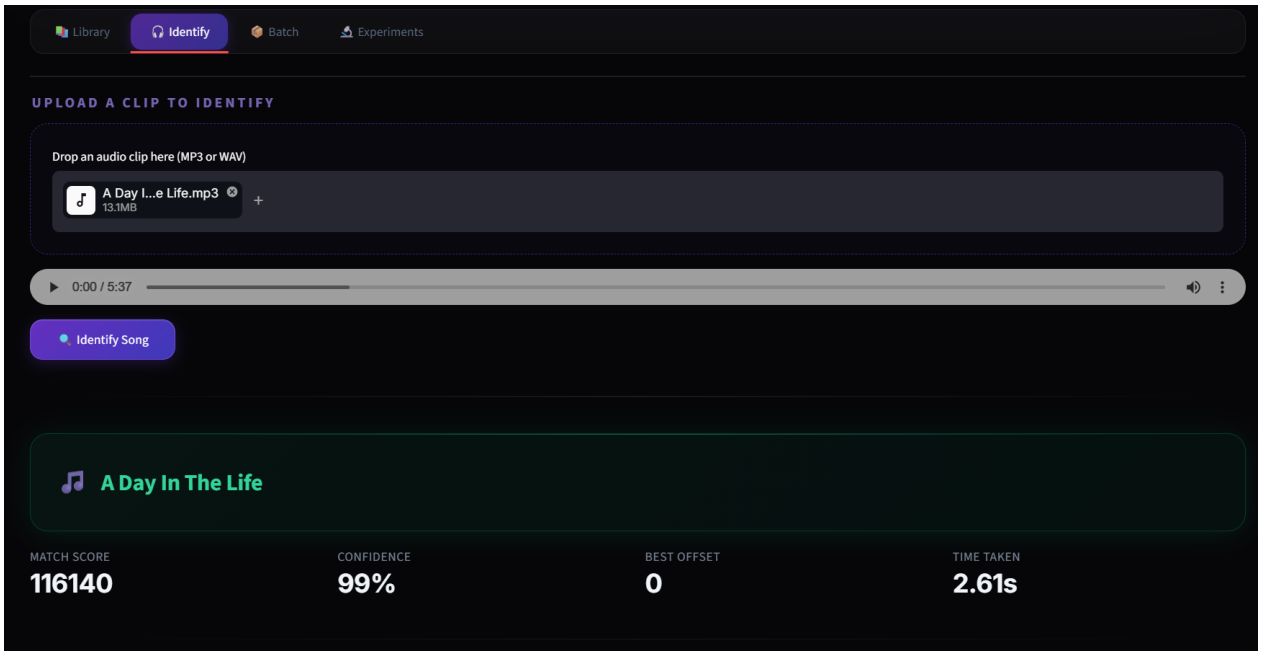


Figure 10 identity tab

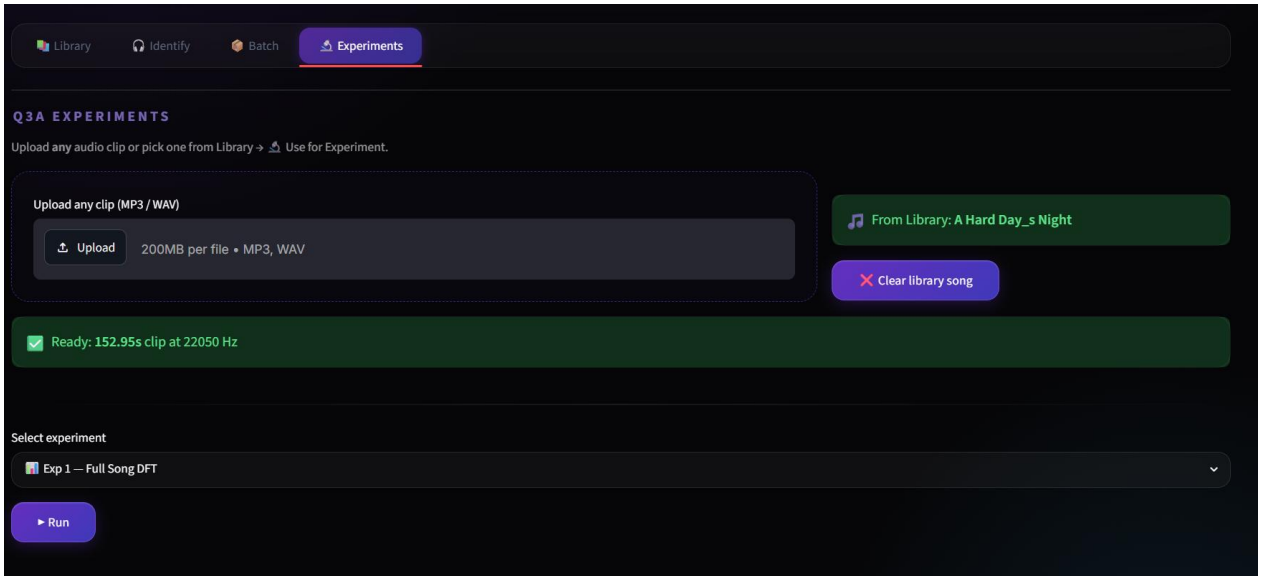


Figure 11: Experiments tab

8. Conclusions

We have successfully implemented a content-based audio identification system modelled on the Shazam algorithm. The key findings are:

- **Spectrograms over DFT:** The STFT retains temporal structure lost in the global DFT, enabling time-localised fingerprinting.
- **Constellation maps:** Retaining only dominant peaks reduces data volume by ~99% while preserving the musically salient features.

- **Pair hashing is decisive:** Encoding (f1, f2, delta_t) reduces collision probability exponentially compared to single-peak hashing, yielding 10-50x higher match scores.
- **Noise robustness:** The system tolerates moderate additive noise (sigma < 0.02) but degrades at higher levels as peak locations are perturbed.
- **Pitch sensitivity:** A shift of even 1 semitone causes failure because absolute frequency bins are stored. Frequency-ratio hashing would fix this.
- **Deployed application:** Sonic Signatures (Hugging Face Spaces) provides a fully interactive demonstration of all Q3A experiments.